

云计算导论Lab3

Cloud Computing Introduction Lab3 Report: Minikube and Kubernetes Basic Practice

Student Name: Linjia Guo

Student ID: 202383910005

Experiment Date: 2026-5-8

Course: Cloud Computing Introduction

1. Experiment Objectives

1. Build and start a single-node Kubernetes cluster using **Minikube**.
 2. Use `kubectl` commands to create, view and manage **Deployments** and **Pods**.
 3. View application logs and access applications via `kubectl proxy`.
 4. Expose internal applications to external access using Kubernetes **Service**.
 5. Use **Labels** to manage and query Kubernetes resources.
 6. Realize application **horizontal scaling** (scale up/scale down).
 7. Complete application **rolling update** and **rollback** operations.
 8. Clean up cluster resources and stop the Minikube environment.
-

2. Experimental Environment

- Operating System: Microsoft Windows 11
 - Container Driver: Docker Desktop
 - Tools: `minikube`, `kubectl`
 - Kubernetes Version: v1.35.1
-

3. Experimental Steps and Results

3.1 Running a Sample Application on Minikube

3.1.1 Start the Minikube Cluster

Run the command to start Minikube with Docker driver:

代码块

```
1 minikube start --driver=docker
```

```
PS C:\Users\27900> minikube start --driver=docker
🐶 Microsoft Windows 11 Home China 25H2 上的 minikube v1.38.1
🔔 根据现有的配置文件使用 docker 驱动程序
👍 在集群中 "minikube" 启动节点 "minikube" primary control-plane
🚧 正在拉取基础镜像 v0.0.50 ...
📦 正在下载 Kubernetes v1.35.1 的预加载文件...
🔄 正在为 "minikube" 重启现有的 docker container ...
🐳 正在 Docker 29.2.1 中准备 Kubernetes v1.35.1...
🔍 正在验证 Kubernetes 组件...
  ▪ 正在使用镜像 gcr.io/k8s-minikube/storage-provisioner:v5
  ▪ 正在使用镜像 docker.io/kubernetesui/metrics-scraper:v1.0.8
  ▪ 正在使用镜像 docker.io/kubernetesui/dashboard:v2.7.0
  ▪ 正在使用镜像 registry.k8s.io/metrics-server/metrics-server:v0.8.1
💡 某些仪表板功能需要 metrics-server 插件。要启用所有功能，请运行：

    minikube addons enable metrics-server

🌞 启用插件： storage-provisioner, metrics-server, default-storageclass, dashboard
🏃 完成！kubectl 现已配置，默认使用 "minikube" 集群和 "default" 命名空间
PS C:\Users\27900> |
```

3.1.2 Check Cluster Status

Verify the running status of Minikube:

代码块

```
1 minikube status
```

```
PS C:\Users\27900> minikube status
minikube
type: Control Plane
host: Running
kubelet: Running
apiserver: Running
kubeconfig: Configured
```

3.1.3 Create a Deployment

Create a `hello-node` Deployment:

代码块

```
1 minikube kubectl -- create deployment hello-node --image=registry.k8s.io/e2e-test-images/agnhost:2.53 -- /agnhost netexec --http-port=8080
```

```
PS C:\Users\27900> minikube kubectl -- create deployment hello-node --image=registry.k8s.io/e2e-test-images/agnhost:2.53 -- /agnhost netexec --http-port=8080
deployment.apps/hello-node created
PS C:\Users\27900>
```

3.1.4 Inspect Deployment and Pod

View Deployment status:

代码块

```
1 minikube kubectl -- get deployments
```

```
PS C:\Users\27900> minikube kubectl -- get deployments
NAME          READY    UP-TO-DATE    AVAILABLE    AGE
hello-node    1/1      1             1            32s
```

View Pod status:

代码块

```
1 minikube kubectl -- get pods
```

```
PS C:\Users\27900> minikube kubectl -- get pods
NAME                                READY   STATUS    RESTARTS   AGE
hello-node-64fc5894d-6rkw2         1/1     Running   0           47s
```

3.1.5 View Application Logs

Check the running logs of the Pod:

代码块

```
1 minikube kubectl -- logs hello-node-xxxx
```

```
PS C:\Users\27900> minikube kubectl -- logs hello-node-64fc5894d-6rkw2
I0509 05:53:59.621476      1 log.go:245] Started HTTP server on port 8080
I0509 05:53:59.623632      1 log.go:245] Started UDP server on port 8081
```

3.1.6 Expose the Application as a Service

Expose Deployment as LoadBalancer Service:

代码块

```
1 minikube kubectl -- expose deployment hello-node --type=LoadBalancer --
  port=8080
```

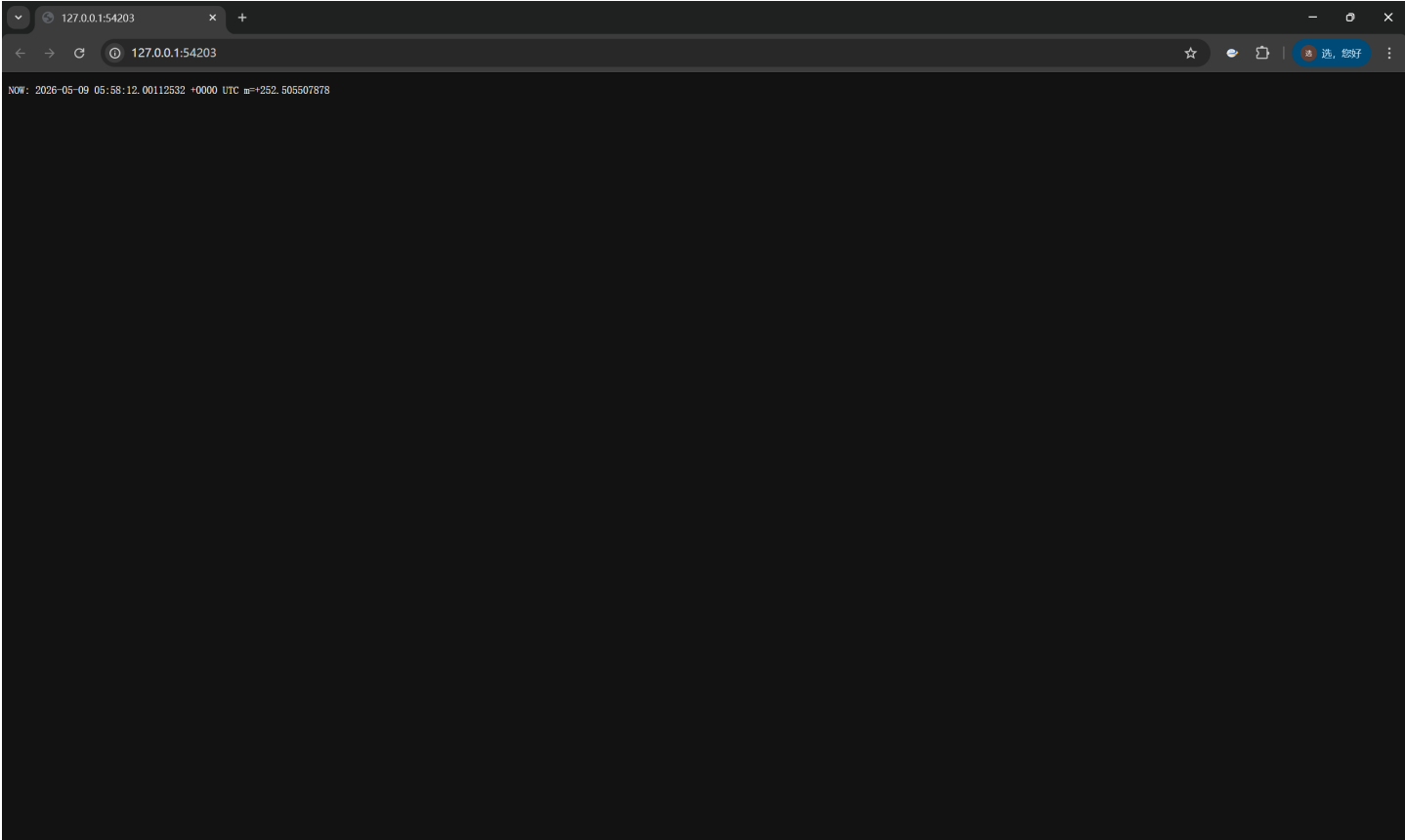
```
PS C:\Users\27900> minikube kubectl -- expose deployment hello-node --type=LoadBalancer --port=8080
service/hello-node exposed
```

3.1.7 Access the Application

Open the application through browser:

代码块

```
1 minikube service hello-node
```



PS C:\Users\27900> minikube service hello-node

NAMESPACE	NAME	TARGET PORT	URL
default	hello-node	8080	http://192.168.49.2:32053

🔗 为服务 `hello-node` 启动隧道。

NAMESPACE	NAME	TARGET PORT	URL
default	hello-node		http://127.0.0.1:54203

🌈 正通过默认浏览器打开服务 `default/hello-node...`
❗ 因为你正在使用 windows 上的 Docker 驱动程序，所以需要打开终端才能运行它。

3.1.8 Enable Metrics-Server Addon

Enable resource monitoring addon:

```
代码块

1 minikube addons enable metrics-server
```

PS C:\Users\27900> minikube addons enable metrics-server

💡 `metrics-server` 是由 Kubernetes 维护的插件。如有任何问题，请在 GitHub 上联系 minikube。
您可以在以下链接查看 minikube 的维护者列表：<https://github.com/kubernetes/minikube/blob/master/OWNERS>

- 正在使用镜像 `registry.k8s.io/metrics-server/metrics-server:v0.8.1`

🌟 启动 '`metrics-server`' 插件

View Pod resource usage:

代码块

```
1 minikube kubectl -- top pods
```

```
PS C:\Users\27900> minikube kubectl -- top pods
NAME                                CPU(cores)   MEMORY(bytes)
hello-node-64fc5894d-6rkw2         1m           36Mi
```

3.1.9 Clean Up Resources

Delete Service and Deployment:

代码块

```
1 minikube kubectl -- delete service hello-node
2 minikube kubectl -- delete deployment hello-node
```

Stop Minikube cluster:

代码块

```
1 minikube stop
```

```
PS C:\Users\27900> minikube kubectl -- delete service hello-node
service "hello-node" deleted from default namespace
PS C:\Users\27900> minikube kubectl -- delete deployment hello-node
deployment.apps "hello-node" deleted from default namespace
PS C:\Users\27900> minikube stop
👉 正在停止节点 "minikube" ...
🔴 正在通过 SSH 关闭“minikube”...
🔴 1 个节点已停止。
```

3.2 Creating a Deployment with kubectl

3.2.1 Check Cluster Nodes

Verify cluster node status:

代码块

```
1 kubectl get nodes
```

```
PS C:\Users\27900> kubectl get nodes
NAME          STATUS    ROLES    AGE   VERSION
minikube      Ready    control-plane   25d   v1.35.1
```

3.2.2 Create kubernetes-bootcamp Deployment

代码块

```
1 kubectl create deployment kubernetes-bootcamp --image=gcr.io/google-samples/kubernetes-bootcamp:v1
```

View Deployment:

代码块

```
1 kubectl get deployments
```

```
PS C:\Users\27900> kubectl create deployment kubernetes-bootcamp --image=gcr.io/google-samples/kubernetes-bootcamp:v1
deployment.apps/kubernetes-bootcamp created
```

View Pod:

代码块

```
1 kubectl get pods
```

```
PS C:\Users\27900> kubectl get pods
NAME                                READY   STATUS    RESTARTS   AGE
kubernetes-bootcamp-67fbdd6b79-nckmt 1/1     Running   0           31s
```

3.2.3 Access Application via kubectl proxy

Start proxy service:

代码块 `kubectl proxy`

```
PS C:\Users\27900> kubectl proxy
Starting to serve on 127.0.0.1:8001
```

Test API connection:

代码块

```
1 Invoke-RestMethod http://localhost:8001/version
```

```
PS C:\Users\27900> Invoke-RestMethod http://localhost:8001/version
```

```
major           : 1
minor           : 35
emulationMajor  : 1
emulationMinor  : 35
minCompatibilityMajor : 1
minCompatibilityMinor : 34
gitVersion      : v1.35.1
gitCommit       : 8fea90b45245ef5c8ba54e7ae044d3e777c22500
gitTreeState    : clean
buildDate       : 2026-02-10T12:53:14Z
goVersion       : go1.25.6
compiler        : gc
platform        : linux/amd64
```

Get Pod name and access application:

代码块

```
1 $POD_NAME = kubectl get pods -o jsonpath="{.items[0].metadata.name}"
2 echo $POD_NAME
3 Invoke-RestMethod
http://localhost:8001/api/v1/namespaces/default/pods/$POD_NAME:8080/proxy/
```

```
PS C:\Users\27900> Invoke-RestMethod "http://localhost:8002/api/v1/namespaces/default/pods/kubernetes-bootcamp-67fbdd6b79-nckmt:8080/proxy/"
Hello Kubernetes bootcamp! | Running on: kubernetes-bootcamp-67fbdd6b79-nckmt | v=1
```


3.3 Viewing Pods and Nodes

3.3.1 Check Running Pods

代码块

```
1 kubectl get pods
```

```
PS C:\Users\27900> kubectl get pods
NAME                                READY   STATUS    RESTARTS   AGE
kubernetes-bootcamp-67fbdd6b79-nckmt 1/1     Running   0           6m7s
```

3.3.2 Describe Pod Details

代码块

```
1 kubectl describe pods
```

```
PS C:\Users\27900> kubectl describe pods
Name:                                kubernetes-bootcamp-67fbdd6b79-nckmt
Namespace:                           default
Priority:                              0
Service Account:                       default
Node:                                 minikube/192.168.49.2
Start Time:                           Sat, 09 May 2026 14:04:05 +0800
Labels:                               app=kubernetes-bootcamp
                                         pod-template-hash=67fbdd6b79
Annotations:                           <none>
Status:                               Running
IP:                                   10.244.0.73
IPs:
IP:                                   10.244.0.73
Controlled By:                         ReplicaSet/kubernetes-bootcamp-67fbdd6b79
Containers:
  kubernetes-bootcamp:
    Container ID:                       docker://9610633f38f521b98cec2733c23b1bcfe636af11e0abacd1ec1aa94123d2ca5a
    Image:                               gcr.io/google-samples/kubernetes-bootcamp:v1
    Image ID:                           docker-pullable://gcr.io/google-samples/kubernetes-bootcamp@sha256:0d6b8ee63bb57c5f5b6156f446b3bc3c143d233037f3a2f00e279c8fcc64af
    Port:                               <none>
    Host Port:                           <none>
    State:                               Running
      Started:                           Sat, 09 May 2026 14:04:06 +0800
    Ready:                               True
    Restart Count:                       0
    Environment:                         <none>
    Mounts:
      /var/run/secrets/kubernetes.io/serviceaccount from kube-api-access-ggdg8 (ro)
Conditions:
  Type                               Status
  PodReadyToStartContainers          True
  Initialized                         True
  Ready                              True
  ContainersReady                    True
  PodScheduled                       True
Volumes:
  kube-api-access-ggdg8:
    Type:                               Projected (a volume that contains injected data from multiple sources)
    TokenExpirationSeconds:             3607
    ConfigMapName:                       kube-root-ca.crt
    Optional:                           false
    DownwardAPI:                       true
QoS Class:                           BestEffort
Node-Selectors:                       <none>
Tolerations:                         node.kubernetes.io/not-ready:NoExecute op=Exists for 300s
                                         node.kubernetes.io/unreachable:NoExecute op=Exists for 300s
Events:
  Type      Reason      Age    From          Message
  -----
  Normal    Scheduled   6m20s  default-scheduler  Successfully assigned default/kubernetes-bootcamp-67fbdd6b79-nckmt to minikube
  Normal    Pulled      6m20s  kubelet        Container image "gcr.io/google-samples/kubernetes-bootcamp:v1" already present on machine and can be accessed by the pod
```

3.3.3 Execute Commands Inside Container

View environment variables:

代码块

```
1 kubectl exec $POD_NAME -- env
```

```
PS C:\Users\27900> $POD_NAME = kubectl get pods -o jsonpath="{.items[0].metadata.name}"
PS C:\Users\27900> kubectl exec $POD_NAME -- env
PATH=/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin
HOSTNAME=kubernetes-bootcamp-67fbdd6b79-nckmt
KUBERNETES_SERVICE_PORT=443
KUBERNETES_SERVICE_PORT_HTTPS=443
KUBERNETES_PORT=tcp://10.96.0.1:443
KUBERNETES_PORT_443_TCP=tcp://10.96.0.1:443
KUBERNETES_PORT_443_TCP_PROTO=tcp
KUBERNETES_PORT_443_TCP_PORT=443
KUBERNETES_PORT_443_TCP_ADDR=10.96.0.1
KUBERNETES_SERVICE_HOST=10.96.0.1
NPM_CONFIG_LOGLEVEL=info
NODE_VERSION=6.3.1
HOME=/root
```

Enter interactive shell:

代码块

```
1 kubectl exec -ti $POD_NAME -- bash
```

```
PS C:\Users\27900> kubectl exec -ti $POD_NAME -- bash
root@kubernetes-bootcamp-67fbdd6b79-nckmt:/# |
```

Test application inside container:

代码块

```
1 curl http://localhost:8080
2 exit
```

```
root@kubernetes-bootcamp-67fbdd6b79-nckmt:/# curl http://localhost:8080
Hello Kubernetes bootcamp! | Running on: kubernetes-bootcamp-67fbdd6b79-nckmt | v=1
root@kubernetes-bootcamp-67fbdd6b79-nckmt:/# exit
```

3.4 Using a Service to Expose Your App

3.4.1 Check Running Application

代码块

```
1  kubectl get pods
2  kubectl get deployments
```

```
PS C:\Users\27900> kubectl get pods
NAME                                READY   STATUS    RESTARTS   AGE
kubernetes-bootcamp-67fbdd6b79-nckmt 1/1     Running   0           8m7s
PS C:\Users\27900> kubectl get deployments
NAME                READY   UP-TO-DATE   AVAILABLE   AGE
kubernetes-bootcamp 1/1     1             1           8m9s
```

3.4.2 Create NodePort Service

代码块

```
1  kubectl expose deployment/kubernetes-bootcamp --type=NodePort --port 8080
```

View Service:

代码块

```
1  kubectl get services
```

```
PS C:\Users\27900> kubectl expose deployment/kubernetes-bootcamp --type=NodePort --port 8080
service/kubernetes-bootcamp exposed
PS C:\Users\27900> kubectl get services
NAME                TYPE        CLUSTER-IP    EXTERNAL-IP   PORT(S)          AGE
kubernetes          ClusterIP   10.96.0.1     <none>        443/TCP          25d
kubernetes-bootcamp NodePort     10.101.204.216 <none>        8080:30674/TCP   1s
```

3.4.3 Access Application via Service

代码块

```
1  minikube service kubernetes-bootcamp --url
```

代码块

```
1 Invoke-RestMethod http://127.0.0.1:xxxxx
```

```
PS C:\Users\27900> minikube service kubernetes-bootcamp --url http://127.0.0.1:55425
! 因为你正在使用 windows 上的 Docker 驱动程序，所以需要打开终端才能运行它。
```

3.4.4 Manage Resources with Labels

Add label to Pod:

代码块

```
1 kubectl label pods $POD_NAME version=v1
```

Query resources by label:

代码块

```
1 kubectl get pods -l app=kubernetes-bootcamp
2 kubectl get services -l app=kubernetes-bootcamp
```

```
PS C:\Users\27900> $POD_NAME = kubectl get pods -o jsonpath="{.items[0].metadata.name}"
PS C:\Users\27900> kubectl label pods $POD_NAME version=v1
pod/kubernetes-bootcamp-67fbdd6b79-nckmt labeled
PS C:\Users\27900> kubectl get pods -l app=kubernetes-bootcamp
NAME                                READY    STATUS    RESTARTS   AGE
kubernetes-bootcamp-67fbdd6b79-nckmt 1/1      Running   0           9m10s
PS C:\Users\27900> kubectl get services -l app=kubernetes-bootcamp
NAME                                TYPE        CLUSTER-IP    EXTERNAL-IP  PORT(S)          AGE
kubernetes-bootcamp                NodePort    10.101.204.216 <none>       8080:30674/TCP   51s
PS C:\Users\27900>
```

3.4.5 Delete Service

代码块

```
1 kubectl delete service -l app=kubernetes-bootcamp
```

Verify deletion:

代码块 `kubectl get services`

```
PS C:\Users\27900> kubectl delete service -l app=kubernetes-bootcamp
service "kubernetes-bootcamp" deleted from default namespace
PS C:\Users\27900> kubectl get services
NAME          TYPE          CLUSTER-IP    EXTERNAL-IP    PORT(S)    AGE
kubernetes    ClusterIP     10.96.0.1     <none>         443/TCP    25d
```

3.5 Running Multiple Instances of Your Application

3.5.1 Create LoadBalancer Service

代码块

```
1 kubectl expose deployment/kubernetes-bootcamp --type=LoadBalancer --port 8080
```

```
PS C:\Users\27900> kubectl expose deployment/kubernetes-bootcamp --type=LoadBalancer --port 8080
service/kubernetes-bootcamp exposed
```

3.5.2 Scale Application to 4 Replicas

代码块

```
1 kubectl scale deployments/kubernetes-bootcamp --replicas=4
```

View scaled Deployment and Pods:

代码块

```
1 kubectl get deployments
2 kubectl get pods -o wide
```

```
PS C:\Users\27900> kubectl expose deployment/kubernetes-bootcamp --type=LoadBalancer --port 8080
service/kubernetes-bootcamp exposed
PS C:\Users\27900> kubectl scale deployments/kubernetes-bootcamp --replicas=4
deployment.apps/kubernetes-bootcamp scaled
PS C:\Users\27900> kubectl get deployments
NAME                READY   UP-TO-DATE   AVAILABLE   AGE
kubernetes-bootcamp 1/4     4            1           9m48s
PS C:\Users\27900> kubectl get pods -o wide
NAME                READY   STATUS              RESTARTS   AGE   IP            NODE           NOMINATED
NODE   READINESS GATES
kubernetes-bootcamp-67fbdd6b79-5l7vp 0/1     ContainerCreating   0         1s    <none>        minikube       <none>
kubernetes-bootcamp-67fbdd6b79-nckmt 1/1     Running             0         9m49s  10.244.0.73   minikube       <none>
kubernetes-bootcamp-67fbdd6b79-sr87l 0/1     ContainerCreating   0         1s    <none>        minikube       <none>
kubernetes-bootcamp-67fbdd6b79-zq9h2 0/1     ContainerCreating   0         1s    <none>        minikube       <none>
```

3.5.3 Verify Load Balancing

Check Service endpoints:

代码块

```
1 kubectl get endpoints kubernetes-bootcamp
```

```
PS C:\Users\27900> kubectl get endpoints kubernetes-bootcamp
Warning: v1 Endpoints is deprecated in v1.33+; use discovery.k8s.io/v1 EndpointSlice
NAME                ENDPOINTS                                                                 AGE
kubernetes-bootcamp 10.244.0.73:8080,10.244.0.74:8080,10.244.0.75:8080 + 1 more... 26s
PS C:\Users\27900>
```

3.5.4 Scale Down to 2 Replicas

代码块

```
1 kubectl scale deployments/kubernetes-bootcamp --replicas=2
```

Verify scale-down result:

代码块

```
1 kubectl get pods
```

```
PS C:\Users\27900> kubectl scale deployments/kubernetes-bootcamp --replicas=2
deployment.apps/kubernetes-bootcamp scaled
PS C:\Users\27900> kubectl get pods
NAME                READY   STATUS              RESTARTS   AGE
kubernetes-bootcamp-67fbdd6b79-5l7vp 1/1     Running             0         28s
kubernetes-bootcamp-67fbdd6b79-nckmt 1/1     Running             0         10m
kubernetes-bootcamp-67fbdd6b79-sr87l 1/1     Terminating       0         28s
kubernetes-bootcamp-67fbdd6b79-zq9h2 1/1     Terminating       0         28s
```

3.6 Performing a Rolling Update

3.6.1 Update Application to v2

代码块

```
1 kubectl set image deployments/kubernetes-bootcamp kubernetes-  
bootcamp=jocatalin/kubernetes-bootcamp:v2
```

Check update status:

代码块

```
1 kubectl rollout status deployments/kubernetes-bootcamp
```

```
PS C:\Users\27900> kubectl set image deployments/kubernetes-bootcamp kubernetes-bootcamp=jocatalin/kubernetes-bootcamp:v2  
deployment.apps/kubernetes-bootcamp image updated  
PS C:\Users\27900> kubectl rollout status deployments/kubernetes-bootcamp  
Waiting for deployment "kubernetes-bootcamp" rollout to finish: 1 out of 2 new replicas have been updated...  
Waiting for deployment "kubernetes-bootcamp" rollout to finish: 1 out of 2 new replicas have been updated...  
Waiting for deployment "kubernetes-bootcamp" rollout to finish: 1 out of 2 new replicas have been updated...  
Waiting for deployment "kubernetes-bootcamp" rollout to finish: 1 old replicas are pending termination...  
Waiting for deployment "kubernetes-bootcamp" rollout to finish: 1 old replicas are pending termination...  
deployment "kubernetes-bootcamp" successfully rolled out
```

Verify updated application:

代码块

```
1 Invoke-RestMethod http://127.0.0.1:xxxxx
```

```
deployment.apps/kubernetes-bootcamp successfully rolled out  
PS C:\Users\27900> minikube service kubernetes-bootcamp --url  
http://127.0.0.1:50148  
! 因为你正在使用 windows 上的 Docker 驱动程序，所以需要打开终端才能运行它。
```

3.6.2 Simulate Failed Update (v10)

代码块

```
1 kubectl set image deployments/kubernetes-bootcamp kubernetes-
```

```
bootcamp=gcr.io/google-samples/kubernetes-bootcamp:v10
```

Check Pod status:

代码块

```
1 kubectl get pods
```

```
PS C:\Users\27900> kubectl set image deployments/kubernetes-bootcamp kubernetes-bootcamp=gcr.io/google-samples/kubernet
s-bootcamp:v10
deployment.apps/kubernetes-bootcamp image updated
PS C:\Users\27900> kubectl get pods
NAME                                READY   STATUS             RESTARTS   AGE
kubernetes-bootcamp-845bc89d75-5fzx2 0/1     ContainerCreating   0           1s
kubernetes-bootcamp-9c9cdbbfc-9k7lc   1/1     Running             0           43s
kubernetes-bootcamp-9c9cdbbfc-lsd9b   1/1     Running             0           41s
PS C:\Users\27900>
```

3.6.3 Roll Back to Stable Version

代码块

```
1 kubectl rollout undo deployments/kubernetes-bootcamp
```

Verify rollback result:

代码块

```
1 kubectl rollout status deployments/kubernetes-bootcamp
```

```
PS C:\Users\27900> kubectl rollout undo deployments/kubernetes-bootcamp
deployment.apps/kubernetes-bootcamp rolled back
PS C:\Users\27900> kubectl rollout status deployments/kubernetes-bootcamp
deployment "kubernetes-bootcamp" successfully rolled out
PS C:\Users\27900>
```

3.6.4 Clean Up All Resources

代码块

```
1 kubectl delete deployments/kubernetes-bootcamp services/kubernetes-bootcamp
```

Verify resource cleanup:


```
代码块
1 kubectl get deployments
2 kubectl get pods
3 kubectl get services
```

```
PS C:\Users\27900> kubectl delete deployments/kubernetes-bootcamp services/kubernetes-bootcamp
deployment.apps "kubernetes-bootcamp" deleted from default namespace
service "kubernetes-bootcamp" deleted from default namespace
PS C:\Users\27900> kubectl get deployments
No resources found in default namespace.
PS C:\Users\27900> kubectl get pods
NAME                                READY   STATUS    RESTARTS   AGE
kubernetes-bootcamp-9c9cdbbfc-9k7lc 1/1     Terminating    0          72s
kubernetes-bootcamp-9c9cdbbfc-lsd9b 1/1     Terminating    0          70s
PS C:\Users\27900> kubectl get services
NAME         TYPE        CLUSTER-IP   EXTERNAL-IP   PORT(S)    AGE
kubernetes   ClusterIP   10.96.0.1    <none>        443/TCP    25d
```

4. Experiment Summary

This experiment comprehensively mastered the basic operation and application deployment of Kubernetes based on Minikube, successfully built a single-node Kubernetes cluster using Minikube, proficiently used kubectl commands to complete the creation, viewing, log query and interactive access of Deployments and Pods, understood the core role of Deployments in managing Pod life cycles and providing automatic fault recovery, realized the external access and load balancing of applications by creating Services of NodePort and LoadBalancer types, mastered the method of organizing and querying resources through Labels, completed the horizontal scaling test of applications from 1 replica to 4 replicas and then to 2 replicas, verified the load balancing effect of Services for back-end Pods, and realized the zero-downtime upgrade of applications through rolling update and the rapid recovery of stable versions through rollback operations, and finally completed the cleaning of cluster resources and the shutdown of Minikube environment, deeply understood the core concepts and working principles of Kubernetes cluster, Deployment, Pod, Service and Label, and mastered the complete process of containerized application deployment, management, scaling and update on Kubernetes.

Screenshot Correspondence

1. Minikube 启动成功界面
2. Minikube 状态查询结果
3. hello-node Deployment 创建成功
4. hello-node Deployment 状态查询
5. hello-node Pod 运行状态
6. hello-node 应用运行日志
7. hello-node Service 创建成功
8. 浏览器访问 hello-node 应用页面
9. metrics-server 插件启用成功
10. Pod 资源占用查询结果
11. Minikube 停止成功
12. Kubernetes 节点状态查询
13. kubernetes-bootcamp Deployment 创建成功
14. kubernetes-bootcamp Pod 运行状态
15. kubectl proxy 启动成功
16. Kubernetes API 版本信息查询
17. 通过代理访问应用成功
18. 运行中 Pod 列表查询
19. Pod 详细信息查询
20. 容器内环境变量查询
21. 进入容器交互式 bash 界面
22. 容器内访问应用测试
23. 应用 Pod 和 Deployment 状态
24. NodePort 类型 Service 创建成功
25. 通过 Service 访问应用成功
26. 按标签查询资源结果
27. Service 删除成功验证
28. LoadBalancer 类型 Service 创建成功
29. 应用扩容至 4 个副本

30. Service 后端端点查询
31. 应用缩容至 2 个副本
32. 应用滚动更新至 v2 成功
33. 应用 v2 版本访问验证
34. 错误版本更新 Pod 异常状态
35. 应用回滚至稳定版本成功
36. 实验资源清理完成验证